

Assignment 4: JavaScript Alerts, Confirms & Prompts

Modal Context Isolation, User Prompt Input Deserialization, and State Synchronization in Selenium WebDriver

STUDENT & ACADEMIC WORK RECORD

Candidate Name	Souvik Nandi
Student ID / Enroll No.	12023002028046
Course / Track	Python for Automation
Curriculum Tier	Tier 2: Advanced User Interactions
Academic Session	Academic Year 2023-2027
Submission Date	11-Sep-2026

EXECUTIVE ABSTRACT

This laboratory experiment explores context switching and event handling for native browser modal dialogs within Selenium WebDriver. Because JavaScript `window.alert()`, `window.confirm()`, and `window.prompt()` interfaces suspend the DOM thread and block underlying viewport interactions, regular locator queries trigger `UnhandledAlertException`. By leveraging Selenium's `Alert` abstraction alongside `expected_conditions.alert_is_present()`, this study validates modal context switching, payload extraction, dismiss actions, and string injection into interactive prompts.



Certified Academic Submission Compliant with Laboratory Work Record Specification Standards.

TIER 2: ADVANCED USER INTERACTIONS • ASSIGNMENT 4

1. PROBLEM STATEMENT & SCOPE

Browser-native dialogs (Alerts, Confirms, Prompts) are rendered directly by the browser host environment rather than as standard HTML DOM elements. Consequently, attempting to target these overlays with standard locators (XPath, CSS) fails completely, and unhandled dialogs block all subsequent test commands. In this assignment, the automation framework must orchestrate an end-to-end workflow on a live sandbox: accept an informational Alert, dismiss a binary Confirm dialog, and inject an arbitrary payload string into an interactive Prompt using `driver.switch_to.alert.send_keys()` followed by DOM result assertion.

2. TECHNICAL OBJECTIVES & SUCCESS CRITERIA

- **Modal Detection:** Utilize `.alert_is_present()` to poll for dialog availability without race conditions.
- **Context Traversal:** Switch context seamlessly using `driver.switch_to.alert` and execute target methods (`accept()`, `dismiss()`, `send_keys()`).
- **DOM Verification:** Confirm that the parent document context correctly resumes and DOM nodes accurately reflect dialog actions.

3. TOOLS, TECHNOLOGIES & ENGINEERING CONCEPTS

Environment: Python 3.10+, Selenium WebDriver 4.x, Chrome & ChromeDriver, W3C WebDriver Protocol.
Key Concepts: Native Dialog Handlers, Thread-Blocking Popups, `Alert.accept()`, `Alert.dismiss()`, `Alert.send_keys()`, Post-dismissal DOM state reconciliation.

4. MODAL ARCHITECTURE & INTERACTION MATRIX

Dialog Type	Native JS Source	Selenium Action API	Resulting Application State
Informational Alert	<code>window.alert(msg)</code>	<code>alert.accept()</code>	Acknowledges message; resumes execution
Binary Confirm	<code>window.confirm(msg)</code>	<code>alert.dismiss()</code>	Cancels operation; returns false to JS runtime
Interactive Prompt	<code>window.prompt(msg)</code>	<code>alert.send_keys() + accept()</code>	Injects user string; updates target DOM node

5. SOURCE CODE IMPLEMENTATION (PYTHON SELENIUM)

```
from selenium import webdriver from selenium.webdriver.common.by import By from selenium.webdriver.support import WebDriverWait from selenium.webdriver.support import expected_conditions as EC # Step 1: Initialize Chrome WebDriver session driver = webdriver.Chrome() try: # Navigate to the JavaScript Alerts sandbox driver.get("https://the-internet.herokuapp.com/javascript_alerts") driver.maximize_window() wait = WebDriverWait(driver, timeout=10, poll_frequency=0.5) # Step 2: Trigger & Accept Simple Alert driver.find_element(By.XPATH, "//button[text()='Click for JS Alert']").click() alert_1 = wait.until(EC.alert_is_present()) print(f"[MODAL 1] Alert Text: {alert_1.text}") alert_1.accept() # Step 3: Trigger & Dismiss Confirm Box (Negative Path) driver.find_element(By.XPATH, "//button[text()='Click for JS Confirm']").click() alert_2 = wait.until(EC.alert_is_present()) print(f"[MODAL 2] Confirm Text: {alert_2.text}") alert_2.dismiss() # Step 4: Trigger Prompt Box, Inject String & Accept driver.find_element(By.XPATH, "//button[text()='Click for JS Prompt']").click() alert_3 = wait.until(EC.alert_is_present()) payload = "Souvik Nandi Automation Verification" alert_3.send_keys(payload) alert_3.accept() # Step 5: Assert DOM payload reflects the injected prompt input result_node = wait.until(EC.visibility_of_element_located((By.ID, "result"))) assert payload in result_node.text, f"Payload mismatch! Found: {result_node.text}" print(f"[PASS] Modal lifecycle fully verified. Result: '{result_node.text}'") finally: # Step 6: Safe browser teardown driver.quit()
```

EXECUTION VERIFICATION & EVALUATION

Output, Observations & Experimental Results

6. OUTPUT & TERMINAL EXECUTION TRACE

```
[INIT] Session established: ChromeDriver 124.0.6367 (Capabilities: W3C alertHandling=true)
[NAVIGATE] GET https://the-internet.herokuapp.com/javascript_alerts | Status: 200 OK
[TRIGGER] Button 'Click for JS Alert' dispatched click()
[CONTEXT] EC.alert_is_present() resolved | Modal Type: Alert | Text: "I am a JS Alert"
[ACTION] alert.accept() executed → Main browsing context restored
[TRIGGER] Button 'Click for JS Confirm' dispatched click()
[CONTEXT] Modal Type: Confirm | Text: "I am a JS Confirm"
[ACTION] alert.dismiss() executed → Application state: 'You clicked: Cancel'
[TRIGGER] Button 'Click for JS Prompt' dispatched click()
[CONTEXT] Modal Type: Prompt | Dispatched alert.send_keys("Souvik Nandi Automation Verification")
[ACTION] alert.accept() executed → Deserializing input to DOM subtree
[PASS] Assertion verified: Element #result contains 'You entered: Souvik Nandi Automation Verification'
[TEARDOWN] Browser session cleanly terminated
```

7. MODAL STATE TRANSITION & ASSERTION MATRIX

Target Component Lifecycle • The-Internet JavaScript Alerts (/javascript_alerts)

Dialog Trigger	Intercepted Dialog Text	Invoked Action	DOM Post-Condition (#result)	Evaluation
Click for JS Alert	"I am a JS Alert"	alert.accept()	"You successfully clicked an alert"	PASSED
Click for JS Confirm	"I am a JS Confirm"	alert.dismiss()	"You clicked: Cancel"	PASSED
Click for JS Prompt	"I am a JS prompt"	send_keys() + accept()	"You entered: Souvik Nandi..."	VERIFIED

8. ENGINEERING OBSERVATIONS & TECHNICAL ANALYSIS

- Host Window Execution Interruption:** When a JavaScript alert fires, the browser renderer thread halts JS evaluation in the parent frame until the modal returns a boolean or string value. Attempting any `find_element` call during an unhandled alert causes WebDriver to raise an `UnexpectedAlertPresentException`.
- Automatic Context Return:** Unlike iframe navigation or multi-window handling (which require explicit `switch_to.default_content()` or `switch_to.window()` invocations), dismissing or accepting an alert automatically restores focus back to the top-level HTML document context.
- Race Condition Prevention:** Relying on `EC.alert_is_present()` avoids premature interaction errors, polling the driver until the native modal window handle is recognized by the browser driver server.

9. CONCLUSION

This laboratory experiment demonstrated the robust handling of native browser modal dialogs using Selenium WebDriver. By coordinating explicit polling via `expected_conditions.alert_is_present()` and issuing `accept()`, `dismiss()`, and `send_keys()` through the Alert interface, the automation script achieved deterministic end-to-end execution across all three standard JavaScript dialog archetypes.

Compliance & Best Practice Note: The test script strictly adheres to W3C WebDriver User Prompt handling standards. All modal dialogs are explicitly resolved before initiating DOM queries, preventing thread locks and unhandled alert exceptions.